Материалът за четиринадесетата лекция от записките:
глава 4.2 (**Изчислими функции**), стр. 175-177,
глава 4.3 (**Недетерминирани машини на Тюринг**), стр. 178-182,
глава 4.4 (**Основни свойства**), стр. 183.

За допълнително четене от учебника на Hopcroft, Motwani, Ullman:
глава 8.4.4 (Nondeterministic Turing Machines), глава 8.6 (Turing Machines and Computers).

Започнахме лекцията с дефиниция за тотална *изчислима функция*

$$f : \Sigma^\star \to \Sigma^\star.$$

Идеята е, че при даден вход $\alpha \in \Sigma^\star$ от началната конфигурация $(\epsilon, q_{start}, \alpha\sqcup)$ достигаме до заключителна конфигурация $(\sqcup^k, q_{accept}, f(\alpha)\sqcup^m)$. Казано по друг начин, в края на изчислението съдържанието на лентата съвпада с функционалната стойност $f(\alpha)$ и главата е подравнена с нейния пръв символ. Добавили сме бленкове, тъй като в рамките на изчислението главата може да минава вляво от първия символ на α и вдясно от последния символ на α. По този начин, първият символ на изхода $f(\alpha)$ не е нужно да е в същата клетка, както първия символ на входа α.

Като пример разгледахме функцията $f : \{0,1\}^\star \to 1 \cdot \{0,1\}^\star$, която добавя 1 в смисъл на естествени числа в двоична бройна система. Считаме, че празната дума представя 0. Също входът може да има водещи нули, но изходът не може (иначе f няма да е функция). Това е пример 4.6 на стр. 176 в записките.

Идеята на алгоритъма е следната. Докато сме в състояние q_0 изтриваме водещите нули (ако има такива). Ако стигнем бленк (входът представя 0), то записваме 1 и приключваме в q_4. Ако стигнем 1 минаваме надясно през остатъка от входа в състояние q_1, докато стигнем първия бленк след входа и се връщаме наляво в състояние q_2. Тук започва същината на добавянето на 1: всички 1-ци в края на входа отдясно наляво се заменят с 0, докато се достигне до първата 0, която се заменя с 1. Разбира се, ако няма такава 0 (т.е. стигнем бленк) то имаме пренос 1 навън. Ако има такава 0, то трябва да минем наляво без да променяме останалата част от входа (състояние q_3) и да подравним главата с първия символ. И в двата случая приключваме в q_4.

За *домашно* реализирайте добавяне на единица в произволна основа $k > 2$. Считайте, че $\Sigma = \{0, 1, \ldots, k-1\}$.

Недетерминирани машини на Тюринг

Една машина на Тюринг $\mathcal{N}$ е недетерминирана, ако функцията на преходите има вида

$$\Delta : Q \setminus \{q_{accept}, q_{reject}\} \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\lhd, \rhd, \square\}).$$

По този начин, ако се намира в незаключително състояние q и съдържанието на текущата клетка върху лентата е x, машината може да се разклонява в няколко конфигурации, по една за всеки елемент на $\Delta(q, x)$.

Едностъпков преход се дефинира така:

$$(\lambda, q, x\rho) \vdash_{\mathcal{N}} (\lambda', q', \rho'),$$

ако съществуват $y \in \Gamma$ и $d \in \{\triangleleft, \triangleright, \square\}$, такива че $(q', y, d) \in \Delta(q, x)$ и $(\lambda, q, x\rho) \vdash_{y,d} (\lambda', q', \rho')$. След това релациите $\vdash^{l}_{\mathcal{N}}$ и $\vdash^{\star}_{\mathcal{N}}$ се дефинират обичайно.

Както при недетерминираните автомати, изчислението на една недетерминирана машина на Тюринг е естествено да се представи чрез дърво. При даден вход $\omega \in \Sigma^{\star}$, коренът на дървото е началната конфигурация $(\epsilon, q_{start}, \omega\sqcup)$ и за всяка достигната конфигурация $(\lambda, q, x\rho)$ нейните синове са всички (λ', q', ρ'), такива че $(\lambda, q, x\rho) \vdash_{\mathcal{N}} (\lambda', q', \rho')$ (на една стъпка). За всеки клон в това дърво (т.е. максимален път с начало в корена) имаме следните четири възможности:

1. Завърша в q_{accept}, т.е. по този клон машината приема думата ω.

2. Завършва в q_{reject}, т.е. по този клон машината отхвърля думата ω.

3. Завършва в $q \in Q \setminus \{q_{accept}, q_{reject}\}$, т.е. стойността на функцията на преходите е $\Delta(q, x) = \emptyset$. Машината спира без резултат по този клон.

4. Зацикля, т.е. никога не завършва.

Както обичайно, третата възможност може да се сведе към втората като се добавят съответни преходи към q_{reject}.

Ще казваме, че машината $\mathcal{N}$ *приема* входната дума ω, $\omega \in L(\mathcal{N})$, ако в дървото на изчислението *има поне успешен клон*, т.е.:

$$L(\mathcal{N}) \;=\; \{\omega \in \Sigma^{\star} \mid (\epsilon, q_{start}, \omega\sqcup) \vdash^{\star}_{\mathcal{N}} (\lambda, q_{accept}, \rho) \text{ за някои } \lambda, \rho \in \Gamma^{\star}\}.$$

По другите клони в дървото на изчислението машината може да има произволно поведение, т.е. за тях имаме всичките четири възможности.

Една недетерминирана машина $\mathcal{N}$ ще наричаме *разрешител*, ако в нейните дървета на изчислението никога не се случват третата и четвъртата възможност, т.е. за произволна входна дума $\omega \in \Sigma^{\star}$ *всеки клон* на изчислението завършва в q_{accept} или в q_{reject}. По този начин, един разрешител *не приема* дума ω, ако всички клонове на изчислението завършват в q_{reject}.

Нашата непосредствена задача ще бъде да покажем как може да се симулира недетерминирана машина на Тюринг с помощта на детерминирана. Идеята ще бъде да обходим по подходящ начин дървото на изчислението. Трябва да внимаваме да не забием в някой безкраен клон и така да пропуснем други клони. Затова обхождането трябва да е задължително в ширина, а не в дълбочина.

По-конкретно, нека е дадена азбука $\Sigma = \{a_0, a_1, \ldots, a_{k-1}\}$, $k \geq 2$, в която буквите са линейно наредени съобразно индексите им.

Ще считаме, че α предшества β в *каноничната наредба* ($\alpha <_{can} \beta$), ако α има по-малка дължина от β или α и β имат равни дължини и α предхожда β в лексикографската наредба ($\alpha <_{lex} \beta$). Тук $\alpha <_{lex} \beta$ означава, че в първата позиция в която се различават, буквата на α е преди буквата на β, по-точно $\alpha = \gamma a_i \alpha'$ и $\beta = \gamma a_j \beta'$, където $i < j$. Лексикографската наредба може да се разглежда и като обичайната наредба на естествените числа, ако считаме че са представени в бройна система с основа k.

При $\Sigma = \{0, 1\}$ каноничната наредба е:

$$\epsilon,\ 0,\ 1,\ 00,\ 01,\ 10,\ 11,\ 000,\ \ldots$$

При $\Sigma = \{0, 1, 2\}$ каноничната наредба е:

$$\epsilon,\ 0,\ 1,\ 2,\ 00,\ 01,\ 02,\ 10,\ 11,\ 12,\ 20,\ 21,\ 22,\ 000,\ \ldots$$

Интуицията тук е, че думите от $\Sigma^\star$ представят върховете на безкрайно k-ично дърво, като всеки връх в дървото е отъждествен с единствения път до него от корена. Така при $\Sigma = \{0, 1\}$ ϵ представя корена, неговите синове са 0 и 1, техните синове са $00, 01$ и $10, 11$, съответно и така нататък. Сега е ясно, че каноничната наредба съответства на обхождането на дървото в ширина. Всеки връх на дървото ще бъде посетен на някоя крайна стъпка.

Лема 1 *Функцията $f_S : \Sigma^\star \to \Sigma^\star$, такава че $f_S(\alpha)$ е непосредствено следващата дума след α в каноничната наредба, е изчислима.*

Доказателство. Модификация на добавянето на 1 (мислим си, че всяка буква е отъждествена с нейния индекс). Разбира се, ще запазим водещите нули и също при пренос наляво от старшата позиция ще добавяме водеща нула, а не единица.

Теорема 2 *Ако езикът L се разпознава от недетерминирана машина на Тюринг $\mathcal{N}$, то съществува детерминирана машина на Тюринг $\mathcal{D}$, която също разпознава L.*

Доказателство. Всеки нов преход в изчислението еднозначно се определя от текущата конфигурация и наредена тройка от $Q \times \Gamma \times \{\triangleleft, \triangleright, \square\}$, която показва по кой клон на дървото тръгваме. Нека $r = |Q| \cdot |\Gamma| \cdot 3$ и да кодираме въпросните наредени тройки с буквите $X = \{x_0, x_1, \ldots, x_{r-1}\}$. Един пример как може да стане това е даден на страница 181 в записките. По-нататък всяко крайно изчисление на машината $\mathcal{N}$ можем да си представяме като крайна дума над азбуката X.

Детерминираната машина $\mathcal{D}$ ще има три ленти: входна лента, управляваща лента и лента за симулация.

На входната лента стои входната дума ω и тя е само за четене, т.е. никога не я променяме.

Управляващата лента съдържа дума от $X^\star$, която показва кой клон на изчислението обхождаме.

Върху лентата за симулация ще изпълняваме машината $\mathcal{N}$.

На всяка итерация от симулацията:

1. Копираме входната лента върху лентата за симулация.

2. Изпълняваме машината $\mathcal{N}$ върху лентата за симулация, съобразно съдържанието на управляващата лента, която показва кои преходи избираме на всяка стъпка.

3. Ако $\mathcal{N}$ завърши в q_{accept} по време на 2., то $\mathcal{D}$ приключва успешно.

4. В противен случай: $\mathcal{N}$ завършва в q_{reject} или в състояние, от което не може да излезе, или се достига бленк върху управляващата лента или буква x_i върху нея, която не съответства на възможен преход. Тогава изтриваме третата лента, изпълняваме функцията f_S от лемата върху управляващата лента и преминаваме на следващата итерация.

Следствие 3 *Ако L се разпознава от недетерминиран* разрешител $\mathcal{N}$*, то съществува детерминиран* разрешител $\mathcal{D}$*, който също разпознава L.*

Доказателство. При фиксирана входна дума $\omega \in \Sigma^\star$, дървото на изчислението на машината $\mathcal{N}$ има крайна разклоненост $\leq r$. Ако допуснем, че това дърво е безкрайно, то от лемата на Кьониг в това дърво има безкраен клон, което противоречи на факта, че $\mathcal{N}$ е разрешител. Така дървото има крайна височина h, която обаче не е известна предварително.

Ще модифицираме симулацията в теоремата, като добавим допълнителен брояч (върху нова лента) на завършилите в q_{reject} изпълнения. Броячът ще анулираме при всяка промяна на дължината h на думата върху управляващата лента. Ако броячът достигне стойност r^h, то е ясно, че $\mathcal{D}$ може да приключи с отхвърляне на входната дума. Както в теоремата, ако $\mathcal{N}$ приеме входната дума, то само тогава $\mathcal{D}$ приключва успешно.

Забележете, че при симулацията имаме експоненциално забавяне, защото в случай на отхвърляне трябва да обходим всичките r^h неуспешни клонове на изчислението на недетерминираната машина. Разбира се, естествено е да приемем, че h е поне колкото дължината на входа, т.е. поне част от клоните прочитат целия вход преди да завършат. Така дори в случай, че всички клони в изчислението на $\mathcal{N}$ са къси, например дължината им зависи линейно от дължината на входа, дължината на съответното изчисление на $\mathcal{D}$ ще зависи експоненциално от дължината на входа.

Твърдение 4 *Ако L_1 и L_2 са полуразрешими (разрешими) езици, то $L_1 \cup L_2$ също е полуразрешим (разрешим) език.*

Доказателство. Нека $L_1 = L(\mathcal{M}_1)$ и $L_2 = L(\mathcal{M}_2)$. Строим нова машина $\mathcal{M}$, която при вход α симулира *паралелно* изчисленията на $\mathcal{M}_1$ и $\mathcal{M}_2$ над α. Забележете, че последователната симулация тук не е подходяща, тъй като можем да зациклим в неуспешно изчисление на едната машина и да се окаже, че изчислението на другата машина е успешно. Състоянията на

$\mathcal{M}$ биха могли да бъдат $Q_1 \times Q_2$ и още някои работни. Най-удобно е $\mathcal{M}$ да има две ленти, една за $\mathcal{M}_1$ и една за $\mathcal{M}_2$. На всяка стъпка $\mathcal{M}$ изпълнява една стъпка върху $\mathcal{M}_1$ и една стъпка върху $\mathcal{M}_2$. Пропускаме останалите детайли. Имаме следните възможности:

1. Ако $\mathcal{M}_1$ приеме α или $\mathcal{M}_2$ приеме α, то $\mathcal{M}$ приема α.
2. Ако $\mathcal{M}_1$ отхвърли α и $\mathcal{M}_2$ отхвърли α, то $\mathcal{M}$ отхвърля α.
3. В противен случай, $\mathcal{M}$ зацикля над α.

Забележете, че 3. се случва, ако една от машините $\mathcal{M}_1$, $\mathcal{M}_2$ зацикля, а другата отхвърля или зацикля.

Разбира се, ако $\mathcal{M}_1$ и $\mathcal{M}_2$ са разрешители, то $\mathcal{M}$ също е разрешител (и тогава 3. никога не се случва).

Твърдение 5 *Ако L_1 и L_2 са полуразрешими (разрешими) езици, то $L_1 \cap L_2$ също е полуразрешим (разрешим) език.*

Доказателство. Нека $L_1 = L(\mathcal{M}_1)$ и $L_2 = L(\mathcal{M}_2)$. Отново строим нова машина $\mathcal{M}$, която при вход α симулира паралелно изчисленията на $\mathcal{M}_1$ и $\mathcal{M}_2$ над α (всъщност, за разлика от предходното твърдение, тук последователната симулация би била подходяща).

1. Ако $\mathcal{M}_1$ приеме α и $\mathcal{M}_2$ приеме α, то $\mathcal{M}$ приема α.
2. Ако $\mathcal{M}_1$ отхвърли α или $\mathcal{M}_2$ отхвърли α, то $\mathcal{M}$ отхвърля α.
3. В противен случай, $\mathcal{M}$ зацикля над α.

Тук 3. се случва, ако една от машините $\mathcal{M}_1$, $\mathcal{M}_2$ зацикля, а другата приема или зацикля.

Както по-горе, ако $\mathcal{M}_1$ и $\mathcal{M}_2$ са разрешители, то $\mathcal{M}$ също е разрешител.

Теорема 6 (Пост) *Един език L е разрешим тогава и само тогава, когато L и $\overline{L}$ са полуразрешими.*

Доказателство. Посоката отляво надясно е очевидна, тъй като всеки разрешим език е полуразрешим и разрешимите езици са затворени относно допълнение. За посоката отдясно наляво, нека $L = L(\mathcal{M}_1)$ и $\overline{L} = L(\mathcal{M}_2)$. Както в горните две твърдения, построяваме нова машина $\mathcal{M}$, която върху вход α извършва паралелна симулация на $\mathcal{M}_1$ и $\mathcal{M}_2$ върху α. Тъй като $L \cup \overline{L} = \Sigma^\star$, то при произволен вход $\alpha \in \Sigma^\star$ или $\mathcal{M}_1$ ще приеме α, което означава че $\mathcal{M}$ трябва да приеме α или $\mathcal{M}_2$ ще приеме α, което означава, че $\mathcal{M}$ трябва да отхвърли α. Други случаи не може да има. Така $\mathcal{M}$ е разрешител и разпознава L.